

1)

When variable-length associated data is allowed, the scheme Enc-then-MAC+AD with CBC mode and any secure MAC does not provide AEAD security because an attacker can modify the associated data without being detected by the MAC.

Suppose that the plaintext is divided into blocks $p_1, p_2, \dots, p_n$, and the associated data is divided into blocks $a_1, a_2, \dots, a_m$, where n and m are variable-length and publicly known. Then the ciphertext is $c_1, c_2, \dots, c_n$, and the MAC is t .

Consider two plaintexts p and p' with the same length n , but with different associated data a and a' , where $|a| \neq |a'|$. We will construct a ciphertext that is valid for both plaintexts, and therefore, the scheme does not provide AEAD security.

Let d be a string of length $|a| - |a'|$. Then, we can construct two different pairs of (ciphertext, MAC) as follows:

$$(c_1, t) || (c_2, t) || \dots || (c_n, t) || \text{MAC}(\text{Enc}(k, d || a || c_1 || c_2 || \dots || c_n))$$

$$(c_1, t) || (c_2, t) || \dots || (c_n, t) || \text{MAC}(\text{Enc}(k, d || a' || c_1 || c_2 || \dots || c_n))$$

The first block of both ciphertexts is the same, and the last block is the same because it is the MAC. The only difference is in the blocks $d || a$ and $d || a'$. However, because the length of d is not fixed and publicly known, the concatenation $d || a || c_1 || c_2 || \dots || c_n$ and $d || a' || c_1 || c_2 || \dots || c_n$ can be the same for different values of d , and therefore, the MAC cannot detect the modification of the associated data.

An attacker can modify the associated data without being detected by the MAC, and the scheme does not provide AEAD security.

2)

Show that if you know the salt s of the Poly-UHF construction, you can efficiently find a collision.

To show that if you know the salt s of the Poly-UHF construction, you can efficiently find a collision, we can proceed as follows:

1. Let H be the universal hash function family used in the Poly-UHF construction.
2. Suppose we have two distinct inputs x and y such that $H(k, s, x) = H(k, s, y)$, where k is the secret key and s is the salt.
3. Then, we have $H(k, s, x) \oplus H(k, s, y) = 0$, where $\oplus$ denotes bitwise XOR.
4. Since XOR is commutative and associative, we can write $H(k, s, x) \oplus H(k, s, y) = H(k, s, x \oplus y)$.

5. Therefore, we have $H(k, s, x \oplus y) = 0$, which means that x and y collide under the hash function $H(k, s, \cdot)$.
6. Since H is a universal hash function family, the probability that x and y collide under $H(k, s, \cdot)$ is at most $1/n$, where n is the size of the input space.
7. However, if we know the salt s , we can try all pairs of distinct inputs x and y in the input space and compute their hash values under $H(k, s, \cdot)$ until we find a collision.
8. Since the input space is fixed and known, and the number of pairs of inputs to check is at most n^2 , this process can be done efficiently in $O(n^2)$ time.
9. Therefore, if we know the salt s of the Poly-UHF construction, we can efficiently find a collision.

3)

Show that if you are allowed to see only the output of Poly-UHF (i.e., the salt remains hidden), on chosen inputs then you can compute the salt.

Suppose the salt has a length of s bits and let $f(k, m)$ be the output of Poly-UHF on key k and message m . Let $m_1, m_2, \dots, m_n$ be n distinct messages of length l bits each, where $n \geq 2^s$.

The attacker can perform the following steps:

1. For each i from 1 to n , compute $f(k, m_i)$.
2. Use a birthday attack to find a collision between the n outputs. Since there are at least 2^s outputs, a collision can be found with high probability using only $O(2^{s/2})$ queries.
3. Let m_a and m_b be the two messages that produce the collision. Since $f(k, m_a) = f(k, m_b)$, we can XOR m_a and m_b to obtain the salt, s .

The reason this attack works is that the salt is used as an input to the pseudorandom function. If we know two inputs that produce the same output, then we can XOR those inputs to obtain the XOR of their inputs to the pseudorandom function.